Apache2 Web Server

Practical Training Documentation

Installation · Configuration · File Sharing · Authentication · Logging

Environment	Details
Platform	Oracle VirtualBox
Operating System	Parrot OS 7.1 Security Edition (KDE)
Web Server	Apache2
Architecture	Host machine → Guest VM → Apache2 → Target

Table of Contents

Section 1

Motive — Why Apache2?

Section 1 — Motive: Why Apache2?

Apache2 is one of the most widely deployed open-source web server packages in the world. It listens for incoming HTTP requests and serves files in response. In cybersecurity — and penetration testing in particular — Apache2 is commonly used to host files that need to be transferred to a target machine during an engagement, making it an essential tool to understand from both the offensive and defensive perspective.

Use Case	Description
File Transfer	Serve scripts or tools to a target machine via HTTP during a pentest engagement
Phishing Simulation	Host a replica login page to test user credential awareness

Use Case	Description
Log Analysis	Apache access logs reveal attack patterns, source IPs, and request history
Access Control	Practice implementing authentication and directory restrictions

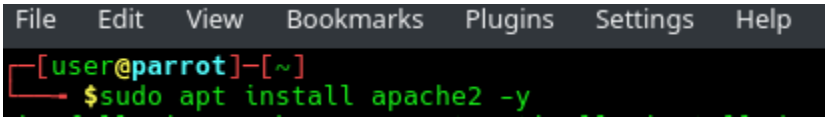
Section 2
Installation and Service Verification

Section 2 — Installation and Service Verification

2.1 — Installing Apache2

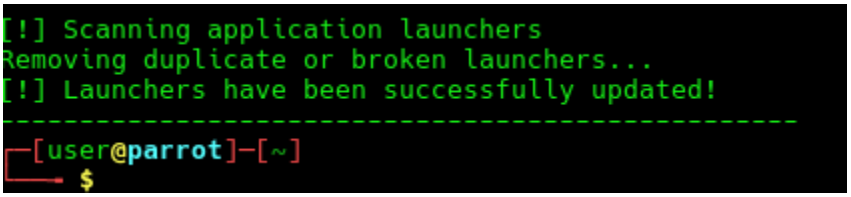
Run the following command to install Apache2 and all required dependencies:

Command — Install Apache2



The package manager will download and install Apache2. The following output confirms a successful installation:

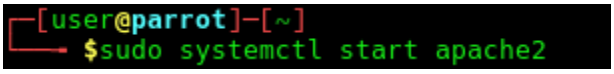
Expected output — Apache2 installed successfully



2.2 — Starting and Enabling the Service

Once installed, start the Apache2 service and enable it to launch automatically on boot:

Command — Start Apache2



Verify that the service is running correctly:

Output — Apache2 service status showing 'active (running)'

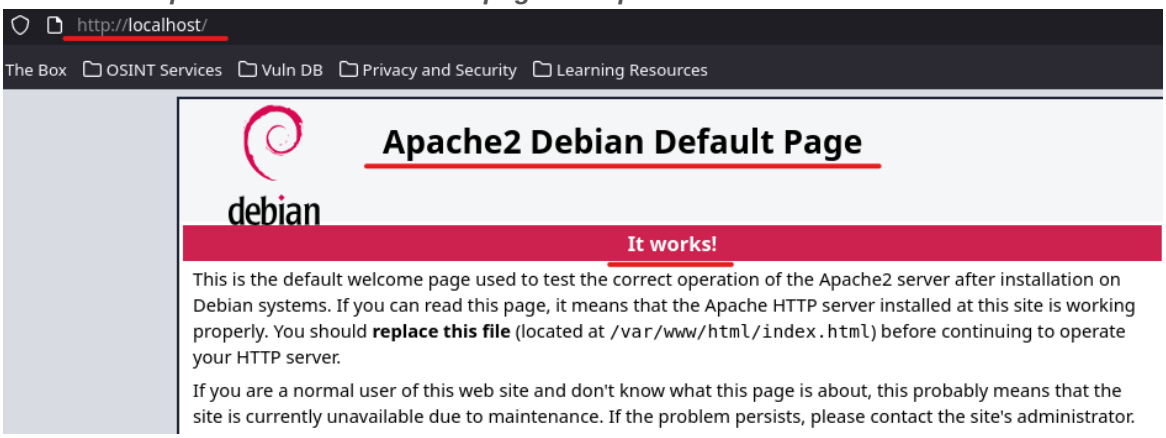
```
┌─[user@parrot]─[~]
└─ $systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
     Active: active (running) since Mon 2026-05-11 21:00:08 UTC; 45s ago
Invocation: 6e398f8e4552429c9539104f5764295c
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 82531 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 82548 (apache2)
      Tasks: 6 (limit: 4583)
     Memory: 18.8M (peak: 19.3M)
        CPU: 43ms
     CGroup: /system.slice/apache2.service
             ├─82548 /usr/sbin/apache2 -k start
             ├─82551 /usr/sbin/apache2 -k start
             ├─82552 /usr/sbin/apache2 -k start
             ├─82553 /usr/sbin/apache2 -k start
             ├─82554 /usr/sbin/apache2 -k start
             └─82555 /usr/sbin/apache2 -k start
```

The green 'active (running)' status confirms Apache2 is live and accepting connections.

2.3 — Browser Verification

Navigate to `http://localhost` in the browser inside your VM. The Apache2 default welcome page confirms the server is running and serving content correctly:

Browser — Apache2 default welcome page at http://localhost



Section 3
Apache2 File Structure

Section 3 — Apache2 File Structure

Before configuring Apache2 or sharing any files, it is essential to understand where its key files and directories are located. Each path serves a specific and important purpose:

Path	Purpose
/var/www/html/	Default web root — files placed here are publicly accessible via HTTP
/etc/apache2/apache2.conf	Main global configuration file — controls core server behaviour
/etc/apache2/sites-available/	Virtual host configuration files — define multiple websites on one server
/var/log/apache2/access.log	Records every incoming HTTP request — essential for traffic analysis
/var/log/apache2/error.log	Records all server errors and warnings — essential for troubleshooting

The /var/www/html/ directory is publicly accessible. Never place sensitive files there unless access controls are in place.

Section 4
Sharing and Downloading Files

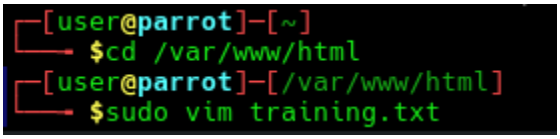
Section 4 — Sharing and Downloading Files

The most practical use of Apache2 in penetration testing is file transfer. The workflow below demonstrates how to place a file on the web server and retrieve it from a target machine — a technique used constantly during real engagements.

4.1 — Creating a File in the Web Root

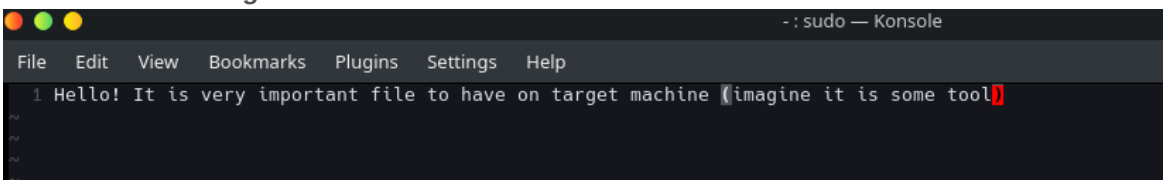
To make a file available for download, it must be placed inside Apache2's web root: `/var/www/html`. The following command opens a new file in that directory using VIM:

Command — Create a new file in /var/www/html using VIM



Add content to the file. When finished, press Esc to exit insert mode, then type :wq to save and exit VIM:

VIM editor — adding content to the file



Confirm the file has been created in the correct location:

Output — File confirmed present in /var/www/html

```
┌─[user@parrot]─[/var/www/html]
└──╼ $ls
index.html  index.nginx-debian.html  training.txt
```

4.2 — Downloading the File from the Target

From the target machine's terminal, use `wget` to download the file directly from the web server:

Command — wget downloading the file from the Apache2 server

```
┌─[user@parrot]─[/]
└──╼ $sudo wget http://localhost/training.txt
--2026-05-11 21:22:39--  http://localhost/training.txt
Resolving localhost (localhost)... ::1
Connecting to localhost (localhost)|::1|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 85 [text/plain]
Saving to: 'training.txt'

training.txt                    100%[=========================================================>]      85  --.-KB/s    in 0s

2026-05-11 21:22:39 (33.0 MB/s) - 'training.txt' saved [85/85]
```

Once downloaded, verify the file contents on the target machine to confirm the transfer was successful:

Output — File contents displayed on the target machine confirming successful transfer

```
┌─[user@parrot]─[/]
└──╼ $ls
bin   dev  home        initrd.img.old  lib32  media  opt   root  sbin  sys  training.txt  var      vmlinuz.old
boot  etc  initrd.img  lib             lib64  mnt    proc  run   srv   tmp  usr           vmlinuz
┌─[user@parrot]─[/]
└──╼ $cat training.txt
Hello! It is very important file to have on target machine (imagine it is some tool)
┌─[user@parrot]─[/]
```

> In a real penetration test scenario, my Parrot VM acts as the attacker-controlled server. The target machine connects to it via HTTP to retrieve tools or payloads.

Section 5
Establishing Basic HTTP Authentication

Section 5 — Establishing Basic HTTP Authentication

Basic HTTP Authentication restricts access to specific directories or files, requiring a valid username and password before content is served. This section demonstrates a complete setup from installation to verification.

5.1 — Installing the Password Utility

The `apache2-utils` package provides the `htpasswd` tool required to manage password files:

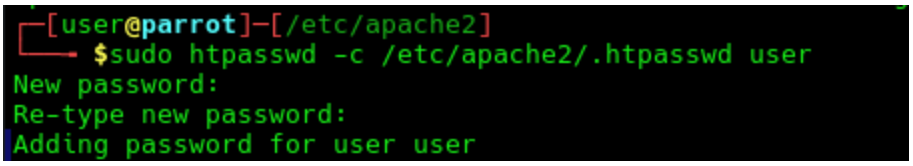
Command — Install apache2-utils

```
┌─[✗]─[user@parrot]─[/]
└──╼ $sudo apt install apache2-utils -y
```

5.2 — Creating the Password File

Create a `.htpasswd` file — a flat-file that stores usernames alongside their encrypted passwords. The `-c` flag creates the file on first use:

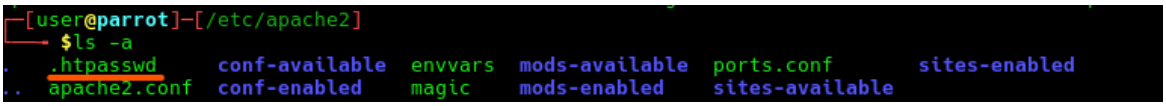
Command — Create .htpasswd file and add a user



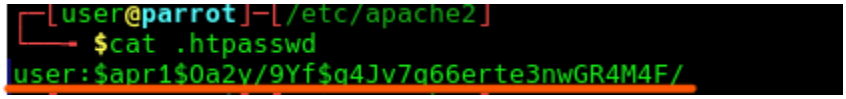
> Only use the -c flag when creating the file for the first time. Using -c again will overwrite the existing file and delete all previously added users.

Verify that the file was created and that it contains the user entry with an encrypted password:

Command — View .htpasswd file contents



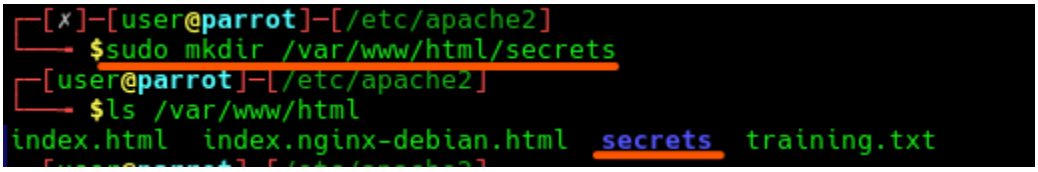
Output — Username and encrypted password hash confirmed



5.3 — Creating the Protected Directory and File

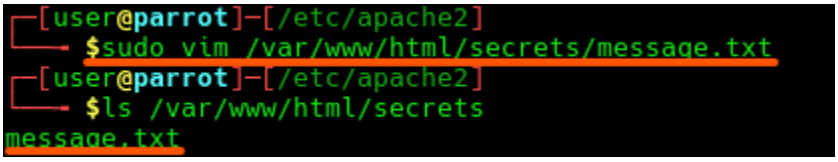
Create a directory and a file inside Apache2's web root (`/var/www/html`) that will be protected by authentication:

Command — Create the protected directory and file



Add a confidential message to the file to test authentication later:

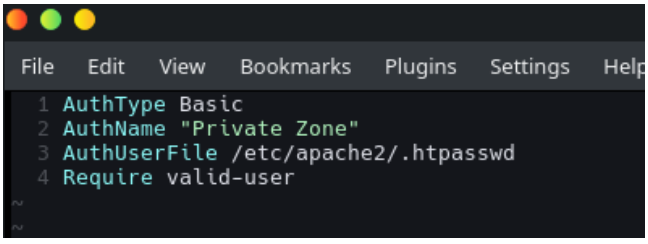
VIM editor — Adding secret content to the protected file



5.4 — Configuring .htaccess

Inside the protected directory, create a `.htaccess` file — a distributed configuration file that controls server behaviour at the directory level. Add the following authentication directives:

.htaccess file — Authentication directives configured



Directive	Meaning
AuthType Basic	Use HTTP Basic Authentication — credentials are Base64 encoded before transmission
AuthName "Private Zone"	The text displayed in the browser's login dialog — can be any descriptive string
AuthUserFile /etc/apache2/.htpasswd	Absolute path to the password file containing valid credentials
Require valid-user	Grant access to any user listed in the .htpasswd file

5.5 — Enabling AllowOverride in Apache Configuration

By default, Apache2 ignores `.htaccess` files entirely. To activate them, open the main configuration file and ensure the `<Directory /var/www/>` block contains `AllowOverride All`:

apache2.conf — AllowOverride All confirmed in the Directory block

```
170 <Directory /var/www/>
171     Options Indexes FollowSymLinks
172     AllowOverride All
173     Require all granted
```

Save the changes and restart Apache2 to apply the new configuration:

Command — Restart Apache2 service

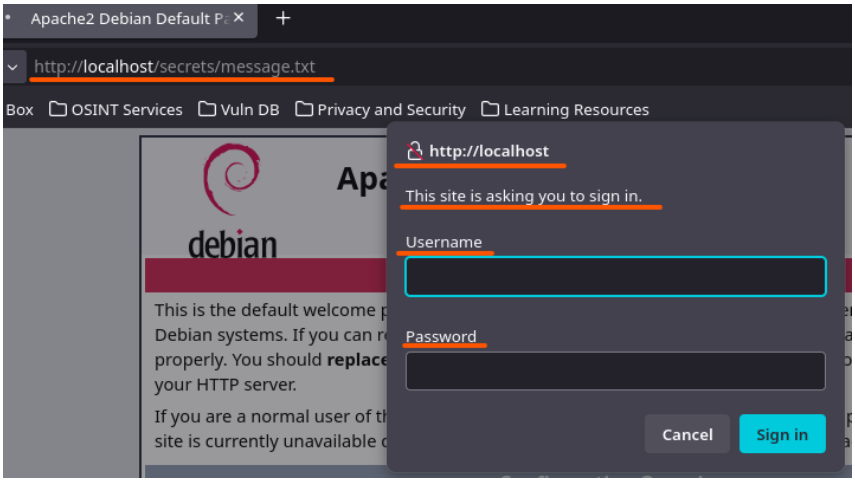
```
┌─[user@parrot]─[/etc/apache2]
└─ $systemctl restart apache2
```

5.6 — Testing Authentication

Via Browser

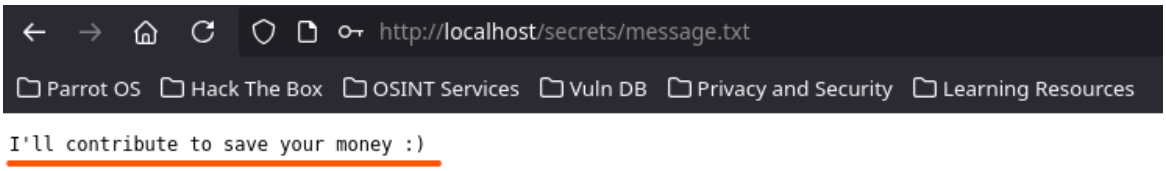
Navigate to the protected directory in the browser. Apache2 will present a login dialog before granting access:

Browser — Login dialog presented when accessing the protected directory



After entering the correct credentials, the protected content is displayed:

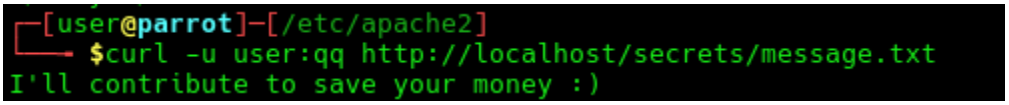
Browser — Protected file content displayed after successful authentication



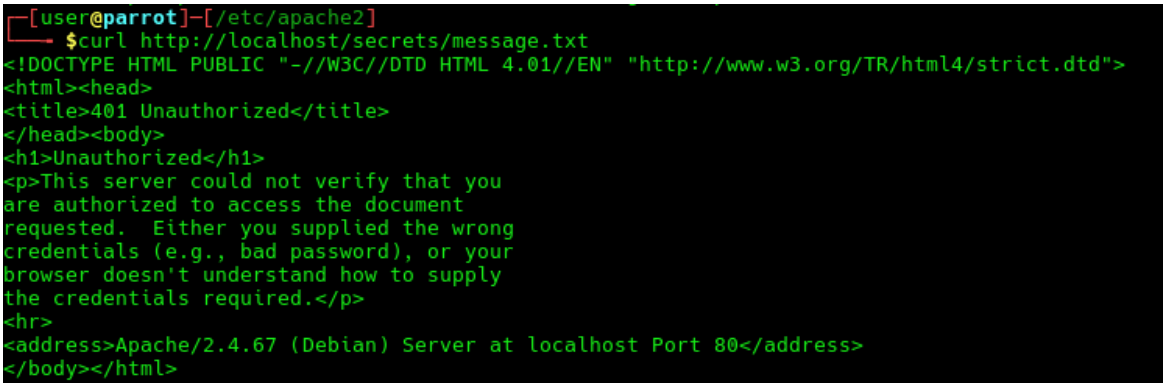
Via Terminal — curl

Test access without credentials using `curl` — the server should respond with a 401 Unauthorized error:

Command — curl request without credentials

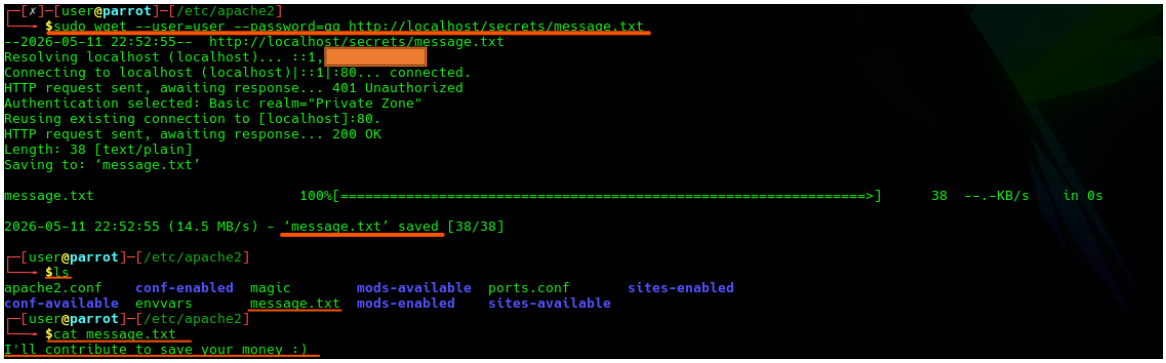


Output — 401 Unauthorized response confirming authentication is enforced



To download the file with valid credentials, pass them using the `--user` and `--password` flags:

Output — curl request with valid credentials successfully retrieving the file



No credentials (401), wrong password (401), correct credentials (200 OK)

Section 6
Error and Access Log Analysis

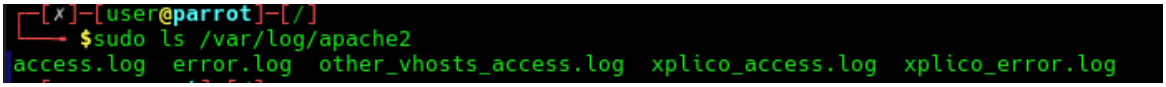
Section 6 — Error and Access Log Analysis

Apache2 maintains detailed logs of all server activity. In cybersecurity, logs are a primary source of evidence during incident response — they reveal who accessed what, when, and from which IP address.

6.1 — Locating the Log Files

All Apache2 logs are stored in `/var/log/apache2/`. List the available log files:

Command — List log files in /var/log/apache2/



6.2 — Reading the Access Log

The access log records every incoming HTTP request. Use `cat` or `tail -f` to read it during an investigation:

Output — Access log showing HTTP request details including IP, method, path, and response code

Each log entry contains: the client IP address, timestamp, HTTP method (GET/POST), requested path, response code, and bytes transferred — all critical data points for security analysis.

Log Field	Meaning
Client IP	Source IP address of the request — used to trace the origin
Timestamp	Exact date and time of the request
HTTP Method	GET (retrieve), POST (submit), PUT (upload) — shows what the client intended
Requested Path	The file or directory that was accessed
Response Code	200 = success, 401 = unauthorized, 403 = forbidden, 404 = not found
Bytes Transferred	Size of the response — useful for detecting large data exfiltration

Section 7
Python3 Web Server — A Lightweight Alternative

Section 7 — Python3 Web Server: A Lightweight Alternative

Python3 includes a built-in HTTP server module that serves any directory instantly with a single command. Unlike Apache2, it requires zero configuration, making it the preferred tool during active penetration testing when speed matters.

Feature	Apache2	Python3 HTTP Server
Setup Time	Requires installation and configuration	Single command — instant start
Configuration	Config files, virtual hosts, modules	No configuration required
Best Used For	Persistent or production web servers	Quick temporary file transfers
Default Port	80	8000 (customisable)

7.1 — Installation

Verify Python3 is installed. On Parrot OS Security Edition it is included by default:

Command — Install/Verify Python3 installation

```
┌─[user@parrot]─[~]
└──╼ $sudo apt install python3 -y
```

7.2 — Starting the Server

Navigate to the directory you want to serve and launch the server with the `-m http.server` module flag:

Command — Start Python3 HTTP server on default port 8000

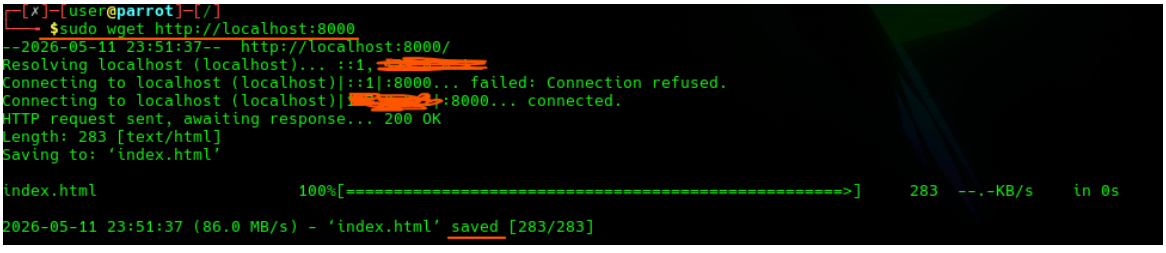
```
┌─[user@parrot]─[/]
└──╼ $python3 -m http.server --directory /home/user/important_dir
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

> The **-m** flag tells Python3 to run a built-in module directly.
>
> http.server is a module that comes pre-installed — no additional packages required.

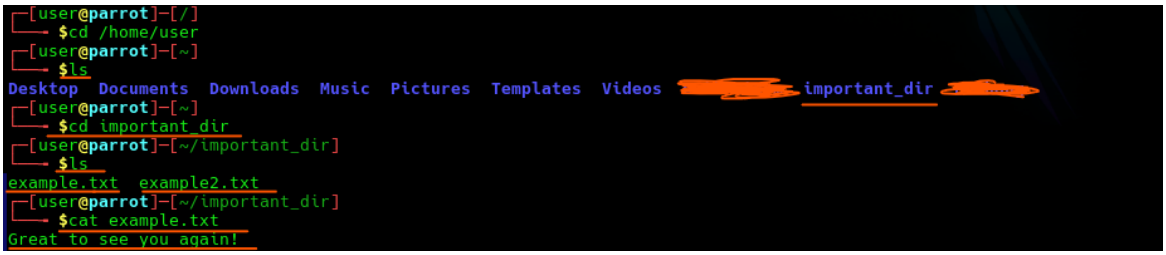
7.3 — Downloading Files from the Terminal

From the target machine's terminal, use `wget` to download files or entire directories from the Python3 server:

Command — wget retrieving files from Python3 web server



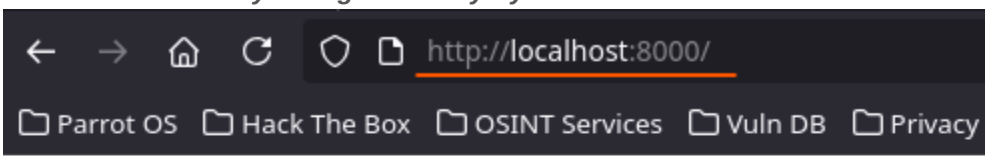
Output — Files successfully downloaded from Python3 server



7.4 — Accessing via Browser

The Python3 server also serves a browsable directory listing. Navigate to the server URL in a browser to view and download files graphically:

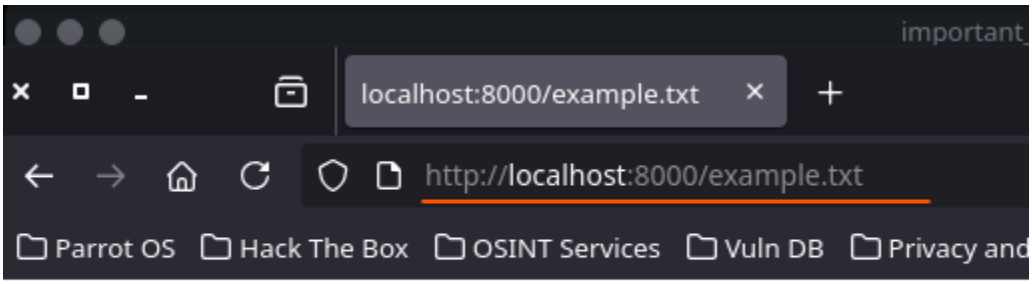
Browser — Directory listing served by Python3 HTTP server



Directory listing for /

- example.txt
- example2.txt

Browser — File contents accessible directly through the Python3 server




```
Great to see you again!
```

The End.